HW-3

Longitudinal MI Training and Plasticity

Date of Submission: 16th April 2026

ECE DEPARTMENT, THE UNIVERSITY OF TEXAS AT AUSTIN

Siyoung Kim

Contents

2.1 BCI Command Delivery **2**
(1) Command Delivery Accuracy and Timeouts . . . 2
(2) Trend Analysis . . . 3
(3) Pairwise Session Comparisons . . . 4

2.2 Discriminability of Features **4**
(1) PSD Feature Extraction . . . 4
(2) Fisher Scores and Top 10 Features . . . 5
(3) Fisher Score Topoplots . . . 5
(4) Best Feature Tracking and Trend Analysis . . . 6
(5) Discriminability–Accuracy Correlation (Subject 2) . . . 7

References **8**

Appendix: Python Code **9**

2.1 BCI Command Delivery

(1) Command Delivery Accuracy and Timeouts

For each online session and run, trial outcomes were classified by parsing the trigger codes: hits were identified by triggers 7693 (LH) and 7703 (RH), while the remaining non-hit endings (7692/7702) were separated into misses and timeouts by examining the task execution duration. Since the BCI protocol enforces a 7-second timeout window, trials ending with a 7692/7702 trigger and lasting $\geq$ 6.9 s were classified as timeouts, while those ending earlier were classified as misses (indicating a wrong command was delivered before timeout). Command delivery accuracy and timeout percentage were computed as:

$$\text{Accuracy} = \frac{\#\text{Hits}}{\#\text{Hits} + \#\text{Misses}}, \qquad \%\text{Timeout} = \frac{\#\text{Timeouts}}{\#\text{Trials}} \tag{1}$$

Table 1 and Table 2 report the per-run values. Figure 1 shows the session-averaged accuracy and timeout percentage across sessions for both subjects.

Table 1: Command delivery accuracy per run for each online session.

	Subject 1			**Subject 2**		
Session	Run 1	Run 2	Run 3	Run 1	Run 2	Run 3
Online-1	0.647	0.643	0.733	0.533	0.450	0.500
Online-2	0.923	0.667	0.900	0.143	0.625	0.545
Online-3	0.692	1.000	1.000	0.800	0.846	0.667
Online-4	0.909	1.000	0.857	1.000	1.000	1.000
Online-5	0.867	1.000	0.824	1.000	1.000	1.000
Online-6	0.941	0.833	0.636	1.000	1.000	1.000

Table 2: Timeout percentage per run for each online session.

	Subject 1			**Subject 2**		
Session	Run 1	Run 2	Run 3	Run 1	Run 2	Run 3
Online-1	0.15	0.30	0.25	0.25	0.00	0.00
Online-2	0.35	0.25	0.50	0.65	0.60	0.45
Online-3	0.35	0.35	0.40	0.25	0.35	0.70
Online-4	0.45	0.40	0.65	0.75	0.55	0.65
Online-5	0.25	0.20	0.15	0.25	0.45	0.65
Online-6	0.15	0.40	0.45	0.25	0.55	0.50

Looking at Figure 1, Subject 2 shows a clear improvement over sessions: accuracy starts near chance ($\sim$49%) and reaches 100% by session 4, staying there through session 6. Subject 1 already starts fairly high ($\sim$67%) and improves somewhat to around 85–90%, though with more variability between runs. Interestingly, the timeout percentage does not seem to decrease for either subject over sessions. This could suggest that the improvement in accuracy is more about the subjects

getting better at producing distinguishable MI patterns, rather than the decoder getting faster at making decisions.

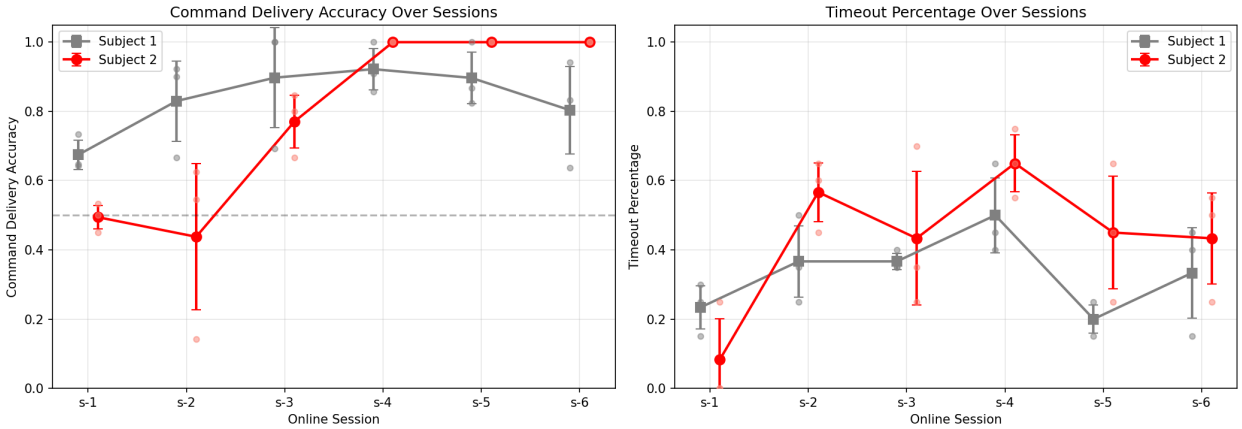

Figure 1: Command delivery accuracy (left) and timeout percentage (right) across online sessions. Individual run data points are shown as scatter, with session means and standard deviations as error bars. The dashed line in the accuracy plot indicates chance level (0.5).

(2) Trend Analysis

To assess the statistical significance of trends in accuracy and timeout percentage across sessions, both Spearman rank correlation and linear regression were performed. Each run was treated as an independent data point ($n = 3$ per subject per session). Results are summarised in Table 3.

Table 3: Trend analysis for accuracy and timeout percentage. Spearman correlation (ρ) and linear regression (R, slope) of session number vs. metric. Significant results ($p < 0.05$) are marked with *.

	Accuracy		**Timeout %**	
Level	ρ (p-value)	Slope (p-value)	ρ (p-value)	Slope (p-value)
Subject 1	0.240 (0.339)	0.025 (0.190)	0.073 (0.774)	0.004 (0.845)
Subject 2	0.914 (<0.001)*	0.127 (<0.001)*	0.263 (0.293)	0.046 (0.148)
Group	0.641 (<0.001)*	0.076 (<0.001)*	0.203 (0.236)	0.025 (0.187)

At the group level, there is a significant positive trend in accuracy ($\rho = 0.641$, $p < 0.001$). This seems to be mostly driven by Subject 2, who has a very strong trend ($\rho = 0.914$, $p < 0.001$). Subject 1 does not show a significant trend on their own ($p = 0.339$), which might be because they already started at a relatively high accuracy, so there was less room to improve.

For the timeout percentage, none of the tests came out significant—neither at the subject nor at the group level. This is a bit surprising, since one might expect timeouts to decrease as subjects get better. It seems like the improvement is specifically in making the right command when a command *is* delivered, not in reducing the number of trials where no command gets through.

(3) Pairwise Session Comparisons

To identify the session at which significant improvement first emerges relative to baseline (session 1), Mann–Whitney U tests were conducted at the group level ($n = 6$ per session: 3 runs $\times$ 2 subjects). Results are shown in Table 4.

Table 4: Pairwise Mann–Whitney U tests comparing each online session to session 1 (group level, $n = 6$ per session).

	Accuracy		**Timeout %**	
Comparison	U	p-value	U	p-value
Session 1 vs 2	13.0	0.485	2.0	0.012*
Session 1 vs 3	2.0	0.013*	2.0	0.012*
Session 1 vs 4	0.0	0.004*	0.0	0.005*
Session 1 vs 5	0.0	0.004*	10.5	0.252
Session 1 vs 6	3.0	0.019*	5.5	0.052

For accuracy, the first significant difference from session 1 appears at session 3 ($p = 0.013$), and it stays significant through sessions 4, 5, and 6. So it looks like about 3 sessions of training are needed before we can see a clear improvement at the group level. The p-value for session 6 ($p = 0.019$) is a bit higher than sessions 4–5 ($p = 0.004$), probably because Subject 1 had a couple of weaker runs in their last session.

The timeout results are harder to interpret. Sessions 2–4 actually show significant *increases* in timeouts compared to session 1, but this goes away in sessions 5 and 6. This might be because subjects were changing their MI strategies during training, which temporarily made it harder for the decoder to reach a decision.

When looking at each subject individually ($n = 3$ runs per session), the Mann–Whitney tests don't have enough power to detect much. Subject 2 shows borderline results from session 4 ($p = 0.064$), and Subject 1 doesn't show any significant differences. With only 3 data points per group, this is not surprising.

2.2 Discriminability of Features

(1) PSD Feature Extraction

Power spectral density (PSD) features were extracted from task execution periods using Welch's method (`scipy.signal.welch`) with a Hamming window of 1 s (512 samples), 50% overlap. PSD values were computed at 14 frequency bins from 4 to 30 Hz with 2 Hz resolution (i.e., at 4, 6, 8, ..., 30 Hz) for all 32 EEG channels, yielding $32 \times 14 = 448$ features per trial. Across all sessions, each session contained 60 trials (30 LH, 30 RH across 3 runs of 20 trials each).

(2) Fisher Scores and Top 10 Features

The Fisher score was computed for each feature as a measure of class discriminability [3]:

$$F = \frac{(\mu_1 - \mu_2)^2}{\sigma_1^2 + \sigma_2^2} \tag{2}$$

where μ_i and σ_i^2 are the mean and variance of the feature values for class i (LH and RH motor imagery, respectively).

Table 5 reports the top 10 features for Subject 2 on the last online session (session 6). Most of the top features fall into two groups: FC6 in the beta band (20–30 Hz), which is over the right frontal/premotor area, and several parietal channels (P8, POz, P4, P3, P7) at 10 Hz, which falls in the mu/alpha range. This makes sense because for left vs. right hand MI, we would expect to see differences in the mu and beta bands over motor areas, as we discussed in class.

Table 5: Top 10 features for Subject 2 on the last online session (session 6), sorted by Fisher score.

Rank	Channel	Frequency (Hz)	Fisher Score
1	FC6	26	1.627
2	P8	10	1.424
3	FC6	24	1.277
4	POz	10	1.251
5	FC6	28	1.226
6	FC6	30	1.157
7	P4	10	1.105
8	P7	10	1.058
9	FC6	20	1.021
10	P3	10	1.020

Looking at stability, the top 10 features change quite a bit from session to session—no single feature stays in the top 10 for all 7 sessions. This is probably because EEG signals are noisy and non-stationary, and the subjects are likely adjusting their MI strategy as they train. That said, for Subject 2 the later sessions do start to converge around FC6 (beta band) and parietal mu-band features, which could indicate that the subject is settling into a more consistent MI approach.

(3) Fisher Score Topoplots

For each session, the Fisher scores of all 14 frequency bands were summed per channel to produce a single discriminability index for each electrode. Figures 2 and 3 show the resulting topoplots.

Subject 1 shows fairly spread-out discriminability patterns that change a lot between sessions. There is no clear progressive trend, which lines up with the fact that their accuracy trend was not significant either.

Subject 2 is more interesting. Starting from sessions 5 and 6, there is a strong hotspot emerging over the right fronto-central/temporal area (around FC6 and T8). This makes sense from what we know about motor imagery: left-hand MI should produce ERD (event-related desynchronization)

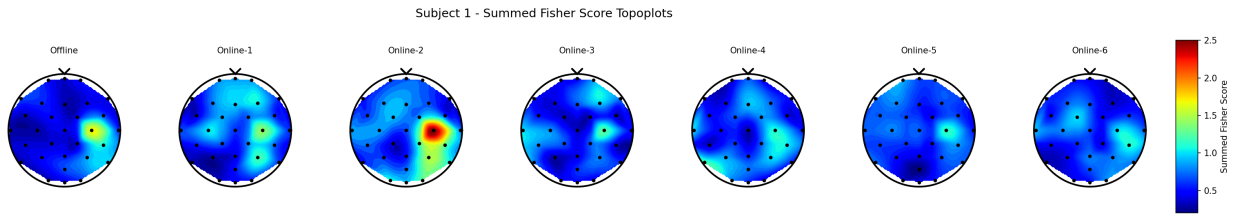

Figure 2: Subject 1: Topoplots of summed Fisher scores across sessions. Discriminability is distributed across central and temporal regions, with some variability but no clear progressive trend.

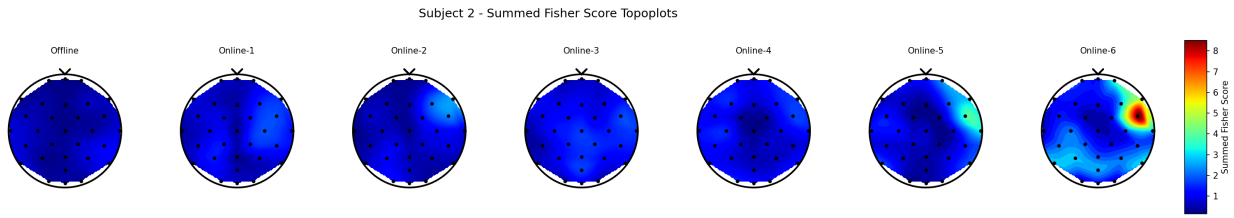

Figure 3: Subject 2: Topoplots of summed Fisher scores across sessions. A clear progressive emergence of discriminability is visible over the right fronto-central/temporal region (FC6, T8), particularly from session 5 onwards.

over the *contralateral* (right) motor cortex [1]. So it looks like Subject 2 learned over time to produce a more focused and stronger ERD pattern on the right side. This kind of progressive focalization has been reported in longitudinal BCI studies like Perdikis et al. [2], and could be a sign of neuroplastic changes from the training.

(4) Best Feature Tracking and Trend Analysis

For each subject, the feature with the highest Fisher score on the last online session was identified and its discriminability was tracked across all six online sessions. Figure 4 shows the results.

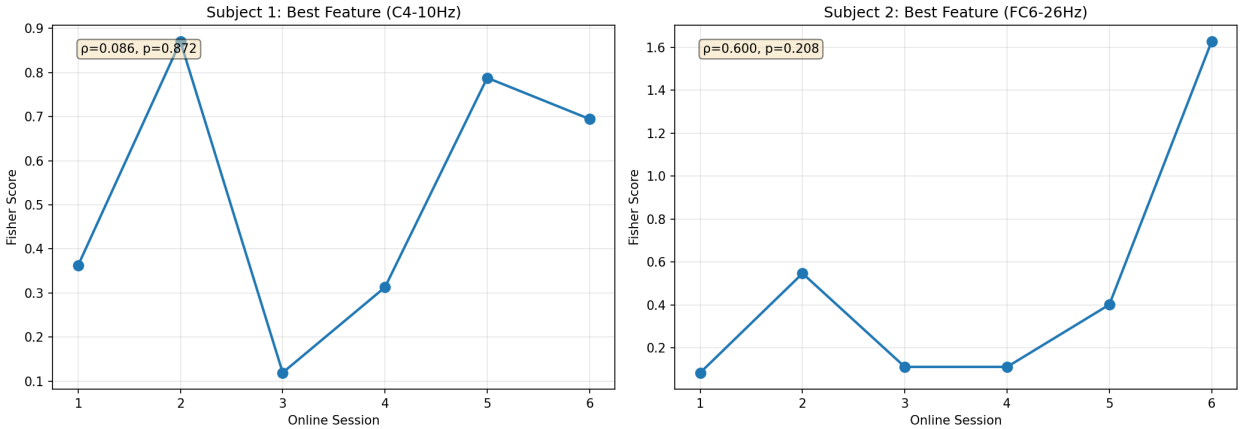

Figure 4: Fisher score of the best feature (identified on session 6) tracked across online sessions. Left: Subject 1 (C4–10 Hz). Right: Subject 2 (FC6–26 Hz).

Subject 1: The best feature is C4–10 Hz, which is in the mu band over the right motor cortex (contralateral to left hand). This is what we would expect for LH vs. RH MI. However, the Fisher score jumps around a lot across sessions with no significant trend ($\rho = 0.086$, $p = 0.872$).

Subject 2: The best feature is FC6–26 Hz (beta band, right side). There is a positive trend ($\rho = 0.600$, $p = 0.208$) but it does not reach significance, likely because we only have 6 data points. The big jump from session 5 to 6 is consistent with what the topoplots show.

At the group level, comparing per-run Fisher scores between session 1 and session 6 gives $U = 5.0$, $p = 0.041$, so there is a significant increase in discriminability when we pool both subjects.

One thing to keep in mind is that this analysis has a bias: since we picked the best feature from the *last* session and looked at how it changed over earlier sessions, we are naturally going to see an increasing trend. It would be better to pick features based on, say, the offline session and then track those. Also, 6 time points is not a lot for trend analysis.

(5) Discriminability–Accuracy Correlation (Subject 2)

The top 10 features from Subject 2's last online session were identified, and their Fisher scores across all online sessions were correlated with the session-averaged command delivery accuracy. Figure 5 shows the scatter plots with Pearson correlation coefficients.

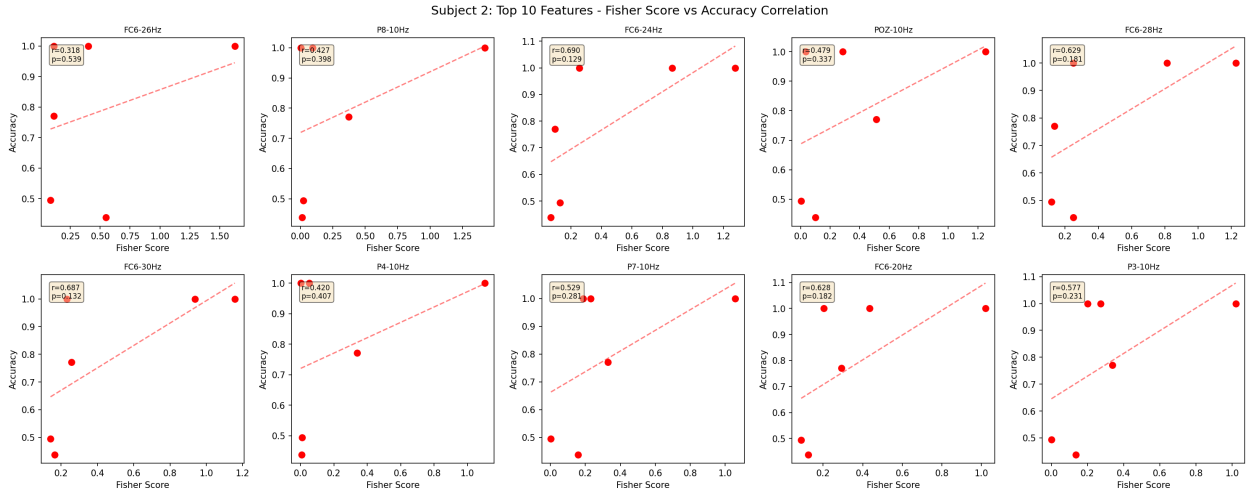

Figure 5: Subject 2: Scatter plots of Fisher score vs. command delivery accuracy across 6 online sessions for each of the top 10 features. Pearson r and p-values are shown. FC6–24 Hz shows a statistically significant Spearman correlation ($\rho = 0.880$, $p = 0.021$).

Table 6 shows the results. Most features have positive correlations with accuracy (r values between 0.3 and 0.7), but since we only have $n = 6$ sessions, only very strong correlations reach significance.

FC6–24 Hz is the only feature with a significant correlation (Spearman $\rho = 0.880$, $p = 0.021$). A few other FC6 beta-band features are close to significance ($p = 0.080$). This is interesting because FC6 is over the right premotor area, and from class we know that the beta band is modulated

Table 6: Correlation between Fisher score and accuracy across sessions for Subject 2's top 10 features. Significant Spearman results ($p < 0.05$) marked with *.

Feature	Pearson r	p-value	Spearman ρ	p-value
FC6–26 Hz	0.318	0.539	0.152	0.774
P8–10 Hz	0.427	0.398	0.273	0.600
FC6–24 Hz	0.690	0.129	0.880	0.021*
POz–10 Hz	0.479	0.337	0.395	0.439
FC6–28 Hz	0.629	0.181	0.759	0.080
FC6–30 Hz	0.687	0.132	0.759	0.080
P4–10 Hz	0.420	0.407	0.273	0.600
P7–10 Hz	0.529	0.281	0.638	0.173
FC6–20 Hz	0.628	0.182	0.759	0.080
P3–10 Hz	0.577	0.231	0.638	0.173

during motor imagery through ERD/ERS [1]. So it seems like as Subject 2's FC6 beta features became more discriminative over sessions, their BCI accuracy also went up, which makes sense.

The parietal 10 Hz features (P8, P4, P3, etc.) have high Fisher scores by session 6 but do not correlate as well with accuracy across sessions. This might be because those features only became really discriminative in the last couple of sessions, so they were not the main factor behind the earlier accuracy improvements.

References

[1] G. Pfurtscheller and F. H. Lopes da Silva, "Event-related EEG/MEG synchronization and desynchronization: basic principles," *Clinical Neurophysiology*, vol. 110, no. 11, pp. 1842–1857, Nov. 1999.

[2] S. Perdikis, L. Tonin, S. Saeedi, C. Schneider, and J. d. R. Millán, "The Cybathlon BCI race: Successful longitudinal mutual learning with two tetraplegic users," *PLOS Biology*, vol. 16, no. 5, p. e2003787, May 2018.

[3] F. Lotte, M. Congedo, A. Lécuyer, F. Lamarche, and B. Arnaldi, "A review of classification algorithms for EEG-based brain–computer interfaces," *Journal of Neural Engineering*, vol. 4, no. 2, pp. R1–R13, Jun. 2007.

Appendix: Python Code

The complete analysis was implemented in Python using `scipy`, `numpy`, and `matplotlib`. Key functions are presented below.

Data Loading and Trial Classification

```python
import scipy.io as sio
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal, stats
from matplotlib.gridspec import GridSpec
import warnings
warnings.filterwarnings('ignore')


# DATA LOADING

def load_subject(filepath, subj_key):
    """Load subject data from .mat file."""
    s = sio.loadmat(filepath)
    return s[subj_key][0, 0]

def load_channel_locations(filepath):
    """Load channel locations for topoplots."""
    ch_locs = sio.loadmat(filepath)
    locs = ch_locs['ch32Locations'][0]
    labels, thetas, radii = [], [], []
    for i in range(len(locs)):
        labels.append(str(locs[i]['labels'][0]))
        thetas.append(float(locs[i]['theta'][0, 0]))
        radii.append(float(locs[i]['radius'][0, 0]))
    return labels, np.array(thetas), np.array(radii)

def get_channel_labels(subj_data):
    """Extract channel labels from subject data."""
    offline = subj_data['offline'][0, 0]
    r1 = offline['run'][0][0]
    header = r1['header'][0, 0]
    ch_raw = header['chLabels']
    return [str(ch_raw[i, 0][0]) if isinstance(ch_raw[i, 0], np.ndarray) else str(ch_raw[i, 0])
            for i in range(ch_raw.shape[0])]

def get_run_data(subj_data, session_type, session_idx=None, run_idx=0):
    """Get EEG data, triggers, and fs for a specific run.
    session_type: 'offline' or 'online'
    session_idx: 0-5 for online sessions (ignored for offline)
    run_idx: 0-2
    """
    if session_type == 'offline':
        sess = subj_data['offline'][0, 0]
        run = sess['run'][0][run_idx]
    else:
```

```python
        online = subj_data['online'][0]
        sess = online[session_idx]
        run = sess['run'][0][run_idx]

    eeg = run['eeg']
    if eeg.shape == ():
        eeg = eeg.item()
    header = run['header'][0, 0]
    fs = int(header['fs'][0, 0])
    triggers = header['triggers'][0, 0]
    typ = triggers['TYP'].flatten()
    pos = triggers['POS'].flatten()
    return eeg, typ, pos, fs

# PART 2.1: BCI COMMAND DELIVERY

TIMEOUT_THRESHOLD_S = 6.9  # seconds

def classify_trials(typ, pos, fs):
    """Classify each online trial as hit, miss, or timeout.
    Returns: n_hits, n_misses, n_timeouts, n_total
    """
    task_start_idx = np.where(np.isin(typ, [7691, 7701]))[0]
    task_end_idx = np.where(np.isin(typ, [7692, 7693, 7702, 7703]))[0]

    n_hits, n_misses, n_timeouts = 0, 0, 0
    n_total = min(len(task_start_idx), len(task_end_idx))

    for i in range(n_total):
        end_typ = typ[task_end_idx[i]]
        if end_typ in [7693, 7703]:  # Hit
            n_hits += 1
        else:  # 7692 or 7702
            duration = (pos[task_end_idx[i]] - pos[task_start_idx[i]]) / fs
            if duration >= TIMEOUT_THRESHOLD_S:
                n_timeouts += 1
            else:
                n_misses += 1

    return n_hits, n_misses, n_timeouts, n_total
```

Listing 1: Loading .mat data and classifying online trials.

```python
% compute_bci_performance
TIMEOUT_THRESHOLD_S = 6.9  # seconds

def classify_trials(typ, pos, fs):
    """Classify each online trial as hit, miss, or timeout.
    Returns: n_hits, n_misses, n_timeouts, n_total
    """
    task_start_idx = np.where(np.isin(typ, [7691, 7701]))[0]
    task_end_idx = np.where(np.isin(typ, [7692, 7693, 7702, 7703]))[0]

    n_hits, n_misses, n_timeouts = 0, 0, 0
    n_total = min(len(task_start_idx), len(task_end_idx))

```

```python
    for i in range(n_total):
        end_typ = typ[task_end_idx[i]]
        if end_typ in [7693, 7703]:  # Hit
            n_hits += 1
        else:  # 7692 or 7702
            duration = (pos[task_end_idx[i]] - pos[task_start_idx[i]]) / fs
            if duration >= TIMEOUT_THRESHOLD_S:
                n_timeouts += 1
            else:
                n_misses += 1

    return n_hits, n_misses, n_timeouts, n_total

def compute_bci_performance(subj_data):
    """Compute accuracy and timeout% for all online sessions.
    Returns:
        acc_per_run: (6, 3) accuracy per run
        to_per_run:  (6, 3) timeout% per run
    """
    acc_per_run = np.zeros((6, 3))
    to_per_run = np.zeros((6, 3))

    for si in range(6):
        for ri in range(3):
            _, typ, pos, fs = get_run_data(subj_data, 'online', si, ri)
            hits, misses, timeouts, total = classify_trials(typ, pos, fs)

            if hits + misses > 0:
                acc_per_run[si, ri] = hits / (hits + misses)
            else:
                acc_per_run[si, ri] = np.nan  # all timeouts

            to_per_run[si, ri] = timeouts / total if total > 0 else np.nan

    return acc_per_run, to_per_run

# --- Load data ---
print("Loading data...")
BASE_DIR = r'C:\Users\c0see\PycharmProjects\ECE374N'
import os

subj1_data = load_subject(os.path.join(BASE_DIR, 'subj1.mat'), 'subj1')
subj2_data = load_subject(os.path.join(BASE_DIR, 'subj2.mat'), 'subj2')
ch_labels = get_channel_labels(subj1_data)
ch_loc_labels, ch_thetas, ch_radii = load_channel_locations(os.path.join(BASE_DIR, '
    ch32Locations.mat'))
print("Computing BCI performance...")
acc1, to1 = compute_bci_performance(subj1_data)
acc2, to2 = compute_bci_performance(subj2_data)
```

Listing 2: Computing command delivery accuracy and timeout percentage per run per session.

```python
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

sessions = np.arange(1, 7)
# Accuracy
```

```python
ax = axes[0]
acc1_mean = np.nanmean(acc1, axis=1)
acc2_mean = np.nanmean(acc2, axis=1)
acc1_std = np.nanstd(acc1, axis=1)
acc2_std = np.nanstd(acc2, axis=1)

# Plot individual runs as scatter
for ri in range(3):
    ax.scatter(sessions - 0.1, acc1[:, ri], c='gray', alpha=0.5, s=20, zorder=3)
    ax.scatter(sessions + 0.1, acc2[:, ri], c='salmon', alpha=0.5, s=20, zorder=3)

ax.errorbar(sessions - 0.1, acc1_mean, yerr=acc1_std, fmt='s-', color='gray',
            label='Subject 1', capsize=4, markersize=8, linewidth=2)
ax.errorbar(sessions + 0.1, acc2_mean, yerr=acc2_std, fmt='o-', color='red',
            label='Subject 2', capsize=4, markersize=8, linewidth=2)
ax.set_xlabel('Online Session')
ax.set_ylabel('Command Delivery Accuracy')
ax.set_title('Command Delivery Accuracy Over Sessions')
ax.set_xticks(sessions)
ax.set_xticklabels([f's-{i}' for i in sessions])
ax.legend()
ax.set_ylim([0, 1.05])
ax.axhline(y=0.5, color='k', linestyle='--', alpha=0.3, label='Chance')
ax.grid(alpha=0.3)

# Timeout %
ax = axes[1]
to1_mean = np.nanmean(to1, axis=1)
to2_mean = np.nanmean(to2, axis=1)
to1_std = np.nanstd(to1, axis=1)
to2_std = np.nanstd(to2, axis=1)

for ri in range(3):
    ax.scatter(sessions - 0.1, to1[:, ri], c='gray', alpha=0.5, s=20, zorder=3)
    ax.scatter(sessions + 0.1, to2[:, ri], c='salmon', alpha=0.5, s=20, zorder=3)

ax.errorbar(sessions - 0.1, to1_mean, yerr=to1_std, fmt='s-', color='gray',
            label='Subject 1', capsize=4, markersize=8, linewidth=2)
ax.errorbar(sessions + 0.1, to2_mean, yerr=to2_std, fmt='o-', color='red',
            label='Subject 2', capsize=4, markersize=8, linewidth=2)
ax.set_xlabel('Online Session')
ax.set_ylabel('Timeout Percentage')
ax.set_title('Timeout Percentage Over Sessions')
ax.set_xticks(sessions)
ax.set_xticklabels([f's-{i}' for i in sessions])
ax.legend()
ax.set_ylim([0, 1.05])
ax.grid(alpha=0.3)

plt.tight_layout()
plt.savefig(os.path.join(BASE_DIR, 'figures', 'fig_2_1_1_accuracy_timeouts.png'), dpi=150,
    bbox_inches='tight')
plt.close()
print("  Saved fig_2_1_1")
# --- Print raw values ---
print("\n=== 2.1.1 Raw Values ===")
```

```python
print("Accuracy per run (rows=sessions, cols=runs):")
print("Subject 1:")
print(np.round(acc1, 3))
print("Subject 2:")
print(np.round(acc2, 3))
print("\nTimeout% per run:")
print("Subject 1:")
print(np.round(to1, 3))
print("Subject 2:")
print(np.round(to2, 3))
```

Listing 3: Plotting accuracy and timeout percentage across sessions with individual run scatter points.

```python
print("\n=== 2.1.2 Trend Analysis ===")

# Flatten: each run is a data point, session number is the IV
# For group level: pool both subjects
session_labels_flat = np.repeat(sessions, 3)  # [1,1,1,2,2,2,...,6,6,6]

# Subject-level: Spearman correlation of session vs accuracy
for name, acc, to in [('Subject 1', acc1, to1), ('Subject 2', acc2, to2)]:
    acc_flat = acc.flatten()
    to_flat = to.flatten()
    valid_acc = ~np.isnan(acc_flat)
    valid_to = ~np.isnan(to_flat)

    r_acc, p_acc = stats.spearmanr(session_labels_flat[valid_acc], acc_flat[valid_acc])
    r_to, p_to = stats.spearmanr(session_labels_flat[valid_to], to_flat[valid_to])
    print(f"  {name} - Accuracy trend: rho={r_acc:.4f}, p={p_acc:.4f}")
    print(f"  {name} - Timeout trend:  rho={r_to:.4f}, p={p_to:.4f}")

# Group level: pool both subjects (n=6 runs per session: 3 from each subject)
acc_group = np.hstack([acc1, acc2])  # (6, 6)
to_group = np.hstack([to1, to2])
session_labels_group = np.repeat(sessions, 6)
acc_group_flat = acc_group.flatten()
to_group_flat = to_group.flatten()

valid = ~np.isnan(acc_group_flat)
r_acc_g, p_acc_g = stats.spearmanr(session_labels_group[valid], acc_group_flat[valid])
valid = ~np.isnan(to_group_flat)
r_to_g, p_to_g = stats.spearmanr(session_labels_group[valid], to_group_flat[valid])
print(f"  Group - Accuracy trend: rho={r_acc_g:.4f}, p={p_acc_g:.4f}")
print(f"  Group - Timeout trend:  rho={r_to_g:.4f}, p={p_to_g:.4f}")

# Also do Page's L test / Jonckheere-Terpstra or linear regression
from scipy.stats import linregress
for name, acc, to in [('Subject 1', acc1, to1), ('Subject 2', acc2, to2)]:
    acc_flat = acc.flatten()
    to_flat = to.flatten()
    valid_acc = ~np.isnan(acc_flat)
    slope, intercept, r, p, se = linregress(session_labels_flat[valid_acc], acc_flat[valid_acc])
    print(f"  {name} - Accuracy linreg: slope={slope:.4f}, R={r:.4f}, p={p:.4f}")
    valid_to = ~np.isnan(to_flat)
```

```python
    slope, intercept, r, p, se = linregress(session_labels_flat[valid_to], to_flat[valid_to])
    print(f"  {name} - Timeout linreg:  slope={slope:.4f}, R={r:.4f}, p={p:.4f}")

valid = ~np.isnan(acc_group_flat)
slope, intercept, r, p, se = linregress(session_labels_group[valid], acc_group_flat[valid])
print(f"  Group - Accuracy linreg: slope={slope:.4f}, R={r:.4f}, p={p:.4f}")
valid = ~np.isnan(to_group_flat)
slope, intercept, r, p, se = linregress(session_labels_group[valid], to_group_flat[valid])
print(f"  Group - Timeout linreg:  slope={slope:.4f}, R={r:.4f}, p={p:.4f}")

# --- 2.1.3: Pairwise session comparison vs session 1 ---
print("\n=== 2.1.3 Pairwise Comparisons (each session vs session 1) ===")

# Group level: session 1 has 6 runs (3 per subject), compare to each other session
# Use Wilcoxon rank-sum (Mann-Whitney U) since samples are independent across sessions
print("\nAccuracy (Group level, Mann-Whitney U):")
s1_acc = acc_group[0, :]  # 6 runs from session 1
for si in range(1, 6):
    s_acc = acc_group[si, :]
    valid1 = ~np.isnan(s1_acc)
    valid2 = ~np.isnan(s_acc)
    if np.sum(valid1) >= 2 and np.sum(valid2) >= 2:
        u_stat, p_val = stats.mannwhitneyu(s1_acc[valid1], s_acc[valid2], alternative='two-sided')
        print(f"  Session 1 vs Session {si+1}: U={u_stat:.1f}, p={p_val:.4f} {'*' if p_val < 0.05 else ''}")

print("\nTimeout% (Group level, Mann-Whitney U):")
s1_to = to_group[0, :]
for si in range(1, 6):
    s_to = to_group[si, :]
    valid1 = ~np.isnan(s1_to)
    valid2 = ~np.isnan(s_to)
    if np.sum(valid1) >= 2 and np.sum(valid2) >= 2:
        u_stat, p_val = stats.mannwhitneyu(s1_to[valid1], s_to[valid2], alternative='two-sided')
        print(f"  Session 1 vs Session {si+1}: U={u_stat:.1f}, p={p_val:.4f} {'*' if p_val < 0.05 else ''}")

# Subject-level pairwise
for name, acc, to in [('Subject 1', acc1, to1), ('Subject 2', acc2, to2)]:
    print(f"\n{name} Accuracy (Mann-Whitney U, n=3 per session):")
    for si in range(1, 6):
        v1 = acc[0, :][~np.isnan(acc[0, :])]
        v2 = acc[si, :][~np.isnan(acc[si, :])]
        if len(v1) >= 2 and len(v2) >= 2:
            u_stat, p_val = stats.mannwhitneyu(v1, v2, alternative='two-sided')
            print(f"    Session 1 vs {si+1}: U={u_stat:.1f}, p={p_val:.4f} {'*' if p_val < 0.05 else ''}")
```

Listing 4: Trend analysis (Spearman correlation and linear regression) and pairwise session comparisons (Mann–Whitney U test).

PSD Feature Extraction and Fisher Score

```python
def extract_task_segments(eeg, typ, pos, fs):
    """Extract EEG segments during task execution periods.
    Returns: list of (segment, class_label) tuples
    class_label: 0 = LH, 1 = RH
    """
    task_starts = np.where(np.isin(typ, [7691, 7701]))[0]
    task_ends = np.where(np.isin(typ, [7692, 7693, 7702, 7703]))[0]

    segments = []
    for i in range(min(len(task_starts), len(task_ends))):
        start_sample = pos[task_starts[i]]
        end_sample = pos[task_ends[i]]
        seg = eeg[start_sample:end_sample, :]
        # Determine class from task start trigger
        class_label = 0 if typ[task_starts[i]] == 7691 else 1  # 0=LH, 1=RH
        segments.append((seg, class_label))
    return segments

def compute_psd_features(segments, fs, freq_range=(4, 30), freq_res=2):
    """Compute PSD features for each trial using Welch's method.
    Returns: features (n_trials, n_features), labels (n_trials,)
    Features = 32 channels x 14 freq bins = 448
    """
    freqs_of_interest = np.arange(freq_range[0], freq_range[1] + freq_res, freq_res)
    # Centers: 4, 6, 8, ..., 30 -> but we want [4-30] with 2Hz resolution
    # That means freq bins at 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30
    # = 14 bins
    n_channels = 32
    n_freqs = len(freqs_of_interest)  # 14

    features = []
    labels = []

    for seg, label in segments:
        n_samples = seg.shape[0]
        if n_samples < fs:  # skip very short segments
            # Use whatever we have, with appropriate nperseg
            nperseg = min(n_samples, fs)
        else:
            nperseg = fs  # 1 second window for 2Hz resolution

        noverlap = nperseg // 2

        trial_features = []
        for ch in range(n_channels):
            f, pxx = signal.welch(seg[:, ch], fs=fs, nperseg=nperseg,
                                  noverlap=noverlap, window='hamming')
            # Extract power at frequencies of interest
            psd_vals = []
            for freq in freqs_of_interest:
                idx = np.argmin(np.abs(f - freq))
                psd_vals.append(pxx[idx])
            trial_features.extend(psd_vals)

```

```
        features.append(trial_features)
        labels.append(label)

    return np.array(features), np.array(labels)

def compute_fisher_scores(features, labels):
    """Compute Fisher score for each feature.
    F = (mu1 - mu2)^2 / (sigma1^2 + sigma2^2)
    """
    class0 = features[labels == 0]
    class1 = features[labels == 1]

    mu0 = np.mean(class0, axis=0)
    mu1 = np.mean(class1, axis=0)
    var0 = np.var(class0, axis=0)
    var1 = np.var(class1, axis=0)

    denom = var0 + var1
    denom[denom == 0] = 1e-12  # avoid division by zero
    fisher = (mu0 - mu1) ** 2 / denom
    return fisher
```

Listing 5: PSD extraction and Fisher score computation.

Topoplot Generation

```

def topoplot_data(fisher_scores, n_channels=32, n_freqs=14):
    """Sum fisher scores across all frequency bands per channel."""
    fisher_2d = fisher_scores.reshape(n_channels, n_freqs)
    return np.sum(fisher_2d, axis=1)  # sum over frequency bands

def plot_topoplot(ax, values, thetas, radii, labels, title='', vmin=None, vmax=None):
    """Plot a simple topoplot using matplotlib."""
    # Convert theta/radius to x,y (EEGLAB convention: theta in degrees, nose up)
    theta_rad = np.deg2rad(thetas)
    x = radii * np.sin(theta_rad)
    y = radii * np.cos(theta_rad)

    # Create grid for interpolation
    from scipy.interpolate import griddata
    grid_x = np.linspace(-0.6, 0.6, 100)
    grid_y = np.linspace(-0.6, 0.6, 100)
    grid_X, grid_Y = np.meshgrid(grid_x, grid_y)

    # Interpolate
    grid_Z = griddata((x, y), values, (grid_X, grid_Y), method='cubic')

    # Mask outside head
    head_radius = 0.55
    mask = np.sqrt(grid_X**2 + grid_Y**2) > head_radius
    grid_Z[mask] = np.nan

    if vmin is None:
        vmin = np.nanmin(grid_Z)
```

```python
    if vmax is None:
        vmax = np.nanmax(grid_Z)

    im = ax.contourf(grid_X, grid_Y, grid_Z, levels=30, cmap='jet', vmin=vmin, vmax=vmax)

    # Draw head
    theta_circle = np.linspace(0, 2*np.pi, 100)
    ax.plot(head_radius * np.cos(theta_circle), head_radius * np.sin(theta_circle), 'k-', linewidth=2)

    # Nose
    ax.plot([0, 0.05, 0], [head_radius, head_radius + 0.05, head_radius], 'k-', linewidth=2)
    ax.plot([0, -0.05, 0], [head_radius, head_radius + 0.05, head_radius], 'k-', linewidth=2)

    # Electrode positions
    ax.scatter(x, y, c='k', s=10, zorder=5)

    ax.set_xlim([-0.7, 0.7])
    ax.set_ylim([-0.65, 0.7])
    ax.set_aspect('equal')
    ax.set_title(title, fontsize=10)
    ax.axis('off')
    return im

# Compute channel-summed fisher scores for topoplots
for subj_name, fisher_list in [('Subject 1', s1_fisher), ('Subject 2', s2_fisher)]:
    fig, axes = plt.subplots(1, 7, figsize=(24, 4))
    fig.suptitle(f'{subj_name} - Summed Fisher Score Topoplots', fontsize=14)

    # Get global min/max for consistent color scale
    all_topo = [topoplot_data(f) for f in fisher_list]
    vmin = min(np.min(t) for t in all_topo)
    vmax = max(np.max(t) for t in all_topo)

    for si in range(7):
        topo_vals = all_topo[si]
        im = plot_topoplot(axes[si], topo_vals, ch_thetas, ch_radii, ch_labels,
                           title=session_names[si], vmin=vmin, vmax=vmax)

    # Colorbar
    fig.subplots_adjust(right=0.92)
    cbar_ax = fig.add_axes([0.93, 0.15, 0.015, 0.7])
    fig.colorbar(plt.cm.ScalarMappable(norm=plt.Normalize(vmin, vmax), cmap='jet'),
                 cax=cbar_ax, label='Summed Fisher Score')

    fname = os.path.join(BASE_DIR, 'figures', f'fig_2_2_3_topoplot_{subj_name.replace(" ", "")}.png')
    plt.savefig(fname, dpi=150, bbox_inches='tight')
    plt.close()
    print(f"  Saved {fname}")
```

Listing 6: Topoplot using matplotlib interpolation.